

Logging Isn't Just print — It's a Table

Summary

Most kdb+ processes log the same way most programs in any language do: a leveled `-1/-2` line to stdout/stderr, maybe with a timestamp, maybe not. That's fine right up until something goes wrong across *several* processes at once — a feed drops, a gateway starts timing out, a pricer starts rejecting quotes — and now the only way to reconstruct what happened is `tail`-ing half a dozen separate log files by hand, lining up timestamps across machines, hoping the clocks agree.

kdb+ already has the right tool for that job sitting right there: a table. If every process ships its log lines into one shared table instead of (or alongside) its own file, "what happened across the fleet in the last five minutes" stops being a grep-and-squint exercise and becomes a `select from logs where timestamp>...` — the same query shape this repo's other two articles already use for trade data.

`sre/logToTab.q` is a small, self-contained logger built around that idea:

- **Write locally, always.** Every call still writes the familiar console banner line and records into an in-memory ring buffer, independent of anything else - this library never makes a process's own visibility *worse*.
- **Forward centrally, if configured.** If a mon address has been given, the same message is also published as one row into a shared `logs` table - the same wire shape a tick.q feed handler uses to publish a trade.
- **Never let monitoring take the process down with it.** A dead or unreachable mon connection degrades to "forwarding silently does nothing" — it must never be the reason the thing being monitored also falls over.

This sits under `sre/` rather than `analytics/` deliberately — it's not a pricing or risk model, it's operational tooling: the same category of code as `analytics/core/`'s tickerplant, just aimed at *the processes themselves* instead of at market data.

Repo

- The code discussed in this article is at: <https://github.com/SpencerFX/q-link>
- To load the library, run the demo scenario, and explore interactively:

```
q ./scripts/initLogging.q
```

- To run the test suite (hard assertions):

```
q ./test/testLogToTab.q
```

- To see what it costs:

```
q ./perf/perfLogToTab.q
```

The banner

The shape of a useful log line isn't a matter of taste — leave out the wrong field and a line stops being useful under pressure. `logs`' schema is built around six things worth knowing about any log line, beyond the message itself:

```
logs:([]
  timestamp:`timestamp$(); sym:`symbol$(); level:`symbol$(); host:`symbol$();
  pid:`int$(); handle:`int$(); user:`symbol$(); mem:`long$(); code:`symbol$();
  message:());
```

Column	Answers
<code>timestamp</code>	<i>When</i> , to the nanosecond, so events across processes can be lined up
<code>sym</code>	<i>Which process</i> — this is <code>.logToTab.procName</code> , playing the same role an instrument symbol plays in a tick, just identifying a process instead of a security
<code>level</code>	<i>How urgent</i> — <code>FATAL ERROR WARN INFO DEBUG</code> , highest severity first
<code>host / pid</code>	<i>Which machine, which instance</i> — the difference between one bad process and a systemic problem
<code>handle</code>	<i>Which connection</i> , if any (<code>.z.w</code>) — the fastest way to trace "which client caused this"
<code>user</code>	<i>Who was on the other end</i> (<code>.z.u</code>) — was this a client query, or an internal timer?
<code>mem</code>	<i>Memory pressure at the time</i> — a slow query and a process approaching its heap limit often show up together
<code>code</code>	<i>A stable, greppable identifier</i> for this specific log statement, independent of the human-readable <code>message</code> text, which is free to change

`code` matters more than it looks. `message` text drifts as code is edited; `code` doesn't have to. A runbook, an alert rule, or a dashboard filter written against `code=`RD_W003` keeps working after someone rewords the message for clarity — the same reason HTTP status codes and this repo's own `PS_W001`-style codes exist independent of their text.

`sym` and `timestamp` first, in that order, isn't an arbitrary convention either — it's the same shape every table in this repo's real-time core (`analytics/core/tp.q`) uses, so `logs` sorts, partitions, and queries the same way a tick table does.

Why the table isn't called `log`

One thing that only shows up by actually trying it: `log` is a reserved q keyword — it's the natural logarithm function, `log exp 1` is `1f`. Attempting a top-level `log:([]...)` doesn't warn, it fails outright:

```
q)log:([] timestamp:`timestamp$(); sym:`symbol$())
'assign
```

`test/testLogToTab.q`'s first assertion pins this down explicitly rather than trusting it stays true: it calls the real `log` function *after* loading `logToTab.q`, to prove the library never shadowed it. The table is `logs` — plural, unreserved, and arguably clearer anyway ("a table of logs" reads better than "a table of log").

Write locally, forward centrally

```
.logToTab.write:[level;code;msg]
t:.z.p;
if[.logToTab.level>=.logToTab.levels level;
  line:"|" sv (_[-3;string t];string .logToTab.procName;string level;string .logToTab.mem[];string code;msg);
  $[.logToTab.levels["WARN"]>=.logToTab.levels level;-2;-1] line
];
.logToTab.tab insert (t;level;code;msg);
time `level`code`msg!(t;level;code;msg)
};

.logToTab.log:[level;code;msg]
logMsg:.logToTab.write[level;code;msg];
if[and[null .logToTab.monHandle;not .logToTab.monAddr~`];.logToTab.connect[.logToTab.monAddr]];
if[not null .logToTab.monHandle;
  row:.logToTab.row logMsg;
  @[[h;msg]neg[h]msg][.logToTab.monHandle];(`upd;`logs;row);
  {[e].logToTab.monHandle:0Ni; .logToTab.write["WARN";LT_W002;"Failed to publish log row to mon process, will retry next call: ",e]
}]
];
logMsg
};
```

Two things worth noticing in `.write`: the `if[.logToTab.level>=...]` guard only wraps the *print*. The `.logToTab.tab insert` line after it runs unconditionally, every time, regardless of the threshold. Raising the console threshold to quiet a noisy terminal doesn't quiet the ring buffer, and — because `.log` forwards unconditionally too — it doesn't quiet what reaches `logs` either. The threshold's whole job is "what's worth scrolling past me right now"; it was never meant to answer "what's worth keeping." Storage is cheap; the five minutes right before an incident, that you didn't think to log at INFO because nothing looked wrong yet, is not something you get to go back and capture retroactively.

The second thing: `.log`'s forwarding is wrapped in a single `@[...]` trap whose failure handler nulls out `monHandle` and calls `.write` again (not `.log` — that would recurse) to report the failure *locally*, then returns normally. Nothing here ever raises past this function. A dashboard being down is not an acceptable reason for a pricing engine to also go down.

Why lazy reconnect, not a timer

The obvious way to keep a dropped mon connection from staying dropped is a background timer that periodically retries — and that's exactly what the [companion implementation in openQ](#) does, `core/tmpbdb.q/core/rdb.q/core/cep.q` all register a 30-second `.util.timer.add` reconnect callback the same way.

`sre/logToTab.q` does something different on purpose: it doesn't own a timer at all.

```
if[and[null .logToTab.monHandle;not .logToTab.monAddr~`];.logToTab.connect[.logToTab.monAddr]];
```

Every `.log` call checks the handle first, and reconnects inline if it's down. The reason isn't laziness — it's that this file is meant to be dropped into *someone else's* process, one that almost certainly already has its own use for `.z.ts`. A logging library that silently claims that slot on load is the kind of surprise that costs an afternoon to track down (openQ's own `utils/timer.q` exists specifically to let several independent pieces of code share `.z.ts` safely — `sre/logToTab.q` sidesteps the whole problem by never needing it).

The tradeoff is real and worth stating plainly: a dropped connection isn't retried until the *next* `.log` call, so the one unlucky message right after a drop pays the reconnect cost inline (in the worst case, an OS-level

connection-refused delay of a couple of seconds — see [perf/perfLogToTab.q](#)'s note on why that path isn't in the timed numbers), rather than a background timer having already reconnected before that message was ever logged. For a library meant to be embedded anywhere, "slightly slower on the first message after an outage" is a better failure mode than "silently took over a resource the host process didn't offer."

The demo

[scripts/initLogging.q](#) runs single-process: it opens its own listening port, defines `logs + upd:insert` in that same process, and has [logToTab.q](#) connect back to itself over a loopback handle. That's a fair stand-in for a separate mon process — the wire protocol (`neg[h](`upd;`logs;row)` landing on a plain `upd:insert`) doesn't know or care that both ends happen to be the same process, and it's the same trick this repo's own [test/testLogToTab.q](#) and [perf/perfLogToTab.q](#) use to stay single-file and dependency-free.

The scenario is a small pricing service's first few seconds: connect, a lagging feed, a rejected quote, a recovery — then the console threshold gets raised to WARN partway through, on purpose, to make the local/forwarded distinction from the previous section visible rather than just asserted:

```
2026.08.24D10:23:30.097321|pricingSvc|INFO|0|PS_I001|connected to rate feed EURUSD
2026.08.24D10:23:30.097495|pricingSvc|WARN|0|PS_W001|rate feed lagging 850ms behind wall clock, buffering
2026.08.24D10:23:30.097520|pricingSvc|ERROR|0|PS_E001|quote for EURUSD rejected: rate is 3200ms stale
2026.08.24D10:23:30.097533|pricingSvc|INFO|0|PS_I002|rate feed caught up, quoting resumed

--- raising console threshold to WARN (forwarding is unaffected) ---

globals left in the workspace: logs, logsSnapshot, ringSnapshot
log count (mon table): 6
log count (local ring buffer): 6
```

Six rows in `logs`, four lines on screen. The DEBUG line from right after startup and the INFO line logged after the threshold was raised are both in that gap — still fully queryable in `logsSnapshot/ringSnapshot`, just never printed. If a real incident review needed "what was happening right before the lagging-feed warning," it's there, even though nobody was watching the console at the time.

Conclusions

The other two articles in this repo ([spread](#), [markout](#)) are about getting a number right — decomposing a quote, recovering a hidden impact curve. This one is about a narrower, more mechanical problem: making sure the *evidence* for figuring either of those out later, when something's actually gone wrong, exists somewhere queryable rather than scattered across N terminal scrollbar buffers. It's less interesting mathematically and more important operationally — nobody debugs a 3am incident by re-deriving a spread decomposition from first principles; they start with `select from logs where timestamp within ... , level in `WARN` `ERROR` `FATAL`` and go from there.

The design choices here are small and mostly defensive: write locally regardless of whether forwarding works, forward regardless of the console threshold, never let a monitoring failure become the monitored process's failure, and don't take a resource (`.z.ts`) the host process might already be using. None of that is exciting. All of it is the difference between a logging library that's safe to drop into a process you don't fully control, and one that isn't.

Appendix: performance

`perf/perfLogToTab.q` times every public function against a live loopback connection — the same setup the demo and test suite use — via `kdb+`'s built-in `\ts` time+space profiler (`perf/perfChk.q`, shared with this repo's other two articles). Console printing is turned off for the timed runs (`.logToTab.setLevel[ `FATAL]`) so these numbers isolate the table-write/publish cost itself, not terminal I/O.

```
q perf/perfLogToTab.q
```

Function	Reps	Avg ms/call
<code>.logToTab.mem</code>	5000	0.0008
<code>.logToTab.row</code>	5000	0.0014
<code>.logToTab.write</code> (local only)	2000	0.001
<code>.logToTab.log</code> (connected)	2000	0.005
<code>.logToTab.log</code> (never configured)	2000	0.0015

`.mem` and `.row` are both sub-microsecond — a `.Q.w[]` call plus arithmetic, and a ten-element tuple build, neither touching a table. `.write` on its own (format, threshold check, ring-buffer insert) costs about the same as `.row` alone, which makes sense — the ring buffer insert is the only table operation in it, and it's a single unkeyed append. `.log` when never configured to forward (`monAddr` still ```) costs `.write`'s price plus one cheap null check — 0.0015ms, barely more than `.write` alone. `.log` when actually connected costs roughly 5x that: 0.005ms, the added cost being the IPC round trip itself (`neg[h] msg`) rather than anything on this side of the wire. Even at the connected number, that's 200,000 log calls/second worth of headroom on this machine — logging was never going to be the bottleneck in a process that also has to price quotes or evaluate a risk grid, and these numbers confirm it isn't, without having to just assume it.

The one path deliberately *not* in this table: the reconnect-attempt cost after a real drop, which is dominated by however long the OS takes to fail (or succeed) the underlying `hopen` — a couple of seconds when nothing is listening, in testing on this machine. That's not a property of this library; it's a property of the network and the address you gave it, and it would swamp every other number here if it were averaged in. `test/testLogToTab.q` exercises and asserts that path; this file doesn't time it.